A pragmatic approach to **reusable research outputs**

Adina S. Wagner[1], J.-B. Poline[2], Michael Hanke[1,3]

[1] Institute of Neuroscience and Medicine, Brain & Behaviour INM-7, Research Center Jülich, Germany
[2] NeuroDataScience - ORIGAMI laboratory, McConnell Brain Imaging Centre, Montreal Neurological Institute-Hospital), Faculty of Medicine and Health Sciences, McGill University, Montreal, Quebec, Canada, and Helen Wills Neuroscience, University of California Berkeley, USA
[3] Institute of Systems Neuroscience, Medical Faculty, Heinrich Heine University Düsseldorf, Germany

FAIRly difficult

Science is an incremental process that produces and builds on **more than journal articles**[1]. Code, data, results, or tools of previous finished or unfinished projects (research outputs) fuel new undertakings.

Reusing research objects allows for reproduction, verification, and extending existing work, evidence synthesis, and minimizing duplicate efforts[2]. The more reusable outputs are, **at any stage of a project**, the better.

The **FAIR principles**[3] center around richly curated metadata to reach maximal reusability. In practice, creating **fully FAIR resources is difficult** in systems that yet lack or fail to incentivize the necessary standards and procedures. But **reusability should be improved nevertheless**.

We highlight **four widely accessible strategies** that can elevate reusability as a byproduct of **pragmatic research data management**, even when compliance to FAIR is not yet possible.

REUSABILITY CHECKLIST

Can you answer the following questions about your research output?

Are all elements of your project unambiguously identifiable?

- ☐ Which version of your script was used to generate the output?
- ☐ What is the precise software environment your analysis was run in?
- ☐ Exactly what data, at which version, have been used?

Can your project speak for itself?

- ☐ Is the parametrization of your analysis "codified" and simply structured?
- ☐ Is relevant processing metadata (e.g. configuration, parametrization) machine-readable, and is used by tools and scripts?
- ☐ Has relevant process metadata been validated via direct use?
- ☐ Can you and others infer what has been done to obtain a result?

Are your project's components modular units?

- ☐ Can someone unfamiliar with your project infer all relevant elements just based on the structure of your project?
- ☐ Is everything that can be reused set up as a stand-alone entity?
- ☐ Does your project declare dependencies to all relevant elements of the processing?

Is your project ready to move?

- ☐ Does your project contain all relevant elements for successful execution (code, data, software environment, ...)?
- ☐ Do your scripts reference project elements with only relative paths?
- ☐ Can you move your project to a different computer, or from local to cloud infrastructure, and execute it without adjustments?

Use **version control for code and data**: Git, DataLad, versioning features of cloud services (e.g., AWS). Use them **continuously, not only as a last step prior to publication**.

Ensure that **everything in your project can be versioned**: enshrine software environments in software containers, write your code into **scripts instead of using GUIs**, include all relevant files in your project.

Capture processing parametrization/invocation (in config files/Make-files, bash scripts bundling all execution steps, automatic execution capture) as an entry point into your project!

Ensure that processing relying on this metadata uses it (by reading configurations from files, etc) **in order to test its validity.**

Modularize when files have different ...

- target **audiences** (raw data vs. results),
- **confidentiality** (personal vs. anonymous),
- levels of **accessibility** (proprietary vs. open formats),
- evolutionary **pace** (factual data vs. ongoing study)

Ensure **independence from system idiosyncrasies** (no hard-coded absolute paths, no infrastructure-specific processing components).

Ensure completeness: all required elements should be a part of your project. **Document everything, and, ideally, automate.**

Portability benefits from previous traits!

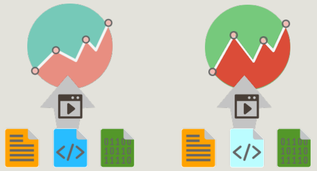
References

[1] Mons, B. (2018). *Data Stewardship for Open Science: Implementing FAIR Principles*. CRC Press. ISBN 9780815348184
[2] Thanos, C. (2017). *Research Data Reusability: Conceptual Foundations, Barriers and Enabling Technologies*. doi.org/10.3390/publications5010002
[3] Wilkinson, M. D. et al., (2016). *The FAIR Guiding Principles for scientific data management and stewardship*. doi.org/10.1038/sdata.2016.18
[4] Kennedy, D. N., et al. (2019). *Everything Matters: The ReproNim Perspective on Reproducible Neuroimaging*. doi.org/10.3389/fninf.2019.00001
[5] Hardwicke, T. E. et al. (2018). *Data availability, reusability, and analytic reproducibility*. doi.org/10.1098/rsos.180448
[6] De Smedt, K. et al. (2020). *FAIR Digital Objects for Science: From Data Pieces to Actionable Knowledge Units*. doi.org/10.3390/publications8020021

Version controlled

All elements involved in the generation of research outputs - from raw or derived data to software - evolve. Changes in these building blocks influence the resulting research output[4].

Identifying precise inputs of a research output is a prerequisite for retracing, trusting, and reproducing its genesis, and is possible when version control systems unambiguously track digital files.



Changed building blocks?

Changed research outputs!

Full transparency
uniquely & un-ambiguously identify digital files.

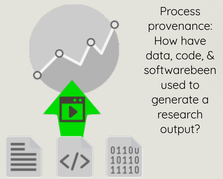
Easier collaboration
Most version control tools facilitate common scientific workflows such as (remote) collaboration and reuse

Full control
Mistakes are not forever anymore. Previous results, code, or data can be brought back to life.

Tidiness
Changes to content are not expressed in additional files, but new versions of the same file.

Actionable

The details of how code, software, or data have been used to generate research outputs (process provenance) are volatile, but required to achieve reproducibility[5].



Capturing this information at the time of its creation, by its expert creators, and basing subsequent processing on it, improves metadata quality and validity even if does not comply to community standards. Ideally, it is simply structured, machine readable, and validated by use. Actionable metadata advances research outputs to FAIR research objects[6].

Reusable research outputs

FAIR digital objects[6]

Common project files

Quality
The max amount of valid process provenance exists at output creation - later reconstruction can be costly or incomplete

Validity
Metadata can proven to be valid when processing based on them results in adaquate outputs. Valid metadata is a basis for reusability.

Reuse & Recompute
Readily available process provenance increases transparency and reproducibility of processing for original authors and reusers.

Modular

Structuring projects into modular units that constitute unambiguous multi-use components (such as raw data, processed data, or software) improves the transparency of a project, & allow individual modules to be recombined and reused easily.

Once you have individual modules, you can declare and link them as dependencies.

```
poster/
  abstract.txt
  a0print.pdf
  poster.svg
  materials/
    infographic/
      caption.txt
      statistics.py
      records.dat
      chart.pdf
    performance/
      figures/
        2020.pdf
        2019.pdf
        2018.pdf
```

Modules can be reused and their purpose inferred from the structure more easily..

Flexibility
Modular units can be reused more easily, in more contexts.

Comprehension
Clear and unblurred structuring aids understanding, especially for non-domain experts and researchers unfamiliar with a project.

Provenance
Research outputs can declare dependencies to individual modules to capture their building blocks.

Portable

When your project is portable, it can move between users or infrastructures and run successfully without adjustments. This requires completeness (every required element should be included in the project and identifyable), documentation (how are processing steps executed, what do they do and what do they need), and automation (see Actionable).

Portable projects can run on your laptop, but also your next laptop, your friends workstation, the cluster, or the cloud.

Infrastructure independence
Even when original compute environments are not accessible, processing is possible.

Reproducibility
Trust and verify outputs by (re-)computing them on different systems.